

Compte Rendu Projet 3,

Romain et Léopold

Partie commune :

Dans le cadre de notre projet, réalisé en binôme avec Leopold, nous avons mis en place une organisation efficace pour travailler ensemble. Cette première partie du compte rendu mettra en lumière notre approche collaborative et détaillera la répartition des tâches que nous avons adoptées. Notre objectif était de maximiser notre productivité et de garantir la réussite de notre projet.

Organisation du travail : Nous avons choisi d'utiliser l'application de communication **Discord** comme principal outil pour notre collaboration. Discord nous a permis de rester en contact de manière régulière et instantanée, favorisant ainsi une communication fluide et une coordination efficace.

Répartition des tâches : Pour assurer une répartition équilibrée des responsabilités, nous avons convenu d'attribuer des tâches spécifiques à chaque membre du binôme. Dans notre cas, **Leopold** s'est occupé de la première partie du projet, notamment des calculs et de la création de la base de données. Cette étape était essentielle, car elle fournissait les informations nécessaires pour créer le reste des graphiques.

Quant à moi, j'ai pris en charge le développement des fonctions **graphique_altitude1()**, **graphique_altitude2()** et **graphique_carte()**, en utilisant les données fournies par Leopold. J'ai pu exploiter ces informations pour créer des visualisations claires et significatives.

Cette approche de répartition des tâches nous a permis d'exploiter au mieux nos compétences respectives et de travailler de manière complémentaire. En travaillant ensemble, nous avons pu avancer de manière efficace et coordonnée tout au long du projet. Cela nous a également permis de partager nos connaissances, renforçant ainsi notre compréhension globale du sujet.

Difficultés rencontrées : La principale difficulté se trouvait dans la fonction **distance_oiseau()** que Léopold devait faire de son côté et elle a été de comprendre comment la boucle for parcourt la base de données pour calculer la distance totale entre les points. Pour cela Léopold m'a appelé pour qu'on règle le problème le plus vite possible et donc on a dû prendre le temps ensemble de bien comprendre le

fonctionnement de la boucle et la façon dont elle utilise les coordonnées géographiques pour calculer la distance entre deux points.

La seconde difficulté que nous avons rencontrée est lors de la création de la fonction `graphique_altitude2()`. Plus précisément, nous avons constaté que l'axe des abscisses du graphique ne correspondait pas aux proportions réelles de la distance parcourue. Cela signifie que la représentation graphique ne reflétait pas correctement la distance réelle entre les points. Nous avons passé du temps à analyser le code et à rechercher une solution pour résoudre ce problème. Nous avons examiné la manière dont les distances étaient calculées et les valeurs associées à l'axe des abscisses. Grâce à nos efforts, nous avons enfin réussi à résoudre cette difficulté dans le temps imparti.

Compte Rendu de Romain :

Dans ce projet, j'ai pris en charge la création de la fonction `graphique_altitude1()`. Cette fonction permet de représenter graphiquement l'évolution du dénivelé point par point entre les points A et B.

Dans cette fonction, j'ai utilisé les altitudes fournies dans le tableau `base`. J'ai extrait les altitudes de chaque point en utilisant une compréhension de liste, comme le montre cette ligne de code : `altitudes = [point[2] for point in base]`. Cela crée une liste contenant uniquement les altitudes des points.

Ensuite, j'ai utilisé la bibliothèque `matplotlib` pour créer le graphique. J'ai utilisé la fonction `plt.plot()` pour tracer la courbe représentant les altitudes. J'ai ajouté des étiquettes pour les axes X et Y à l'aide des fonctions `plt.xlabel()` et `plt.ylabel()`. Enfin, j'ai ajouté un titre au graphique avec la fonction `plt.title()`.

En ce qui concerne les difficultés rencontrées, l'une des principales difficultés que j'ai rencontrées est la manipulation des données du tableau `base`. Il aurait été crucial de s'assurer que les altitudes sont correctement associées aux points correspondants. Donc cette erreur dans la correspondance des données a faussé les résultats du graphique.

En plus de la fonction `graphique_altitude1()`, j'ai également créé les fonctions `graphique_altitude2()` et `graphique_carte()`. Voici un bref aperçu de ce que font ces fonctions :

La fonction `graphique_altitude2()` permet de représenter graphiquement l'évolution du dénivelé en fonction de la distance parcourue. Elle utilise les indices des points pour représenter la distance, et calcule les altitudes relatives par rapport au point de départ. Le graphique obtenu affiche ainsi comment l'altitude varie par rapport à la distance parcourue, fournissant une visualisation intéressante de la progression du dénivelé tout au long du trajet.

La fonction `graphique_carte()` quant à elle permet de tracer le trajet sur une carte en utilisant les coordonnées géographiques (latitude et longitude) des points. Elle utilise les listes des latitudes et des longitudes extraites du tableau `base`. En utilisant la

bibliothèque `matplotlib`, je lui ai appliqué des styles de ligne différents pour chaque segment de ligne reliant les points et des couleurs différentes également en bonus, ce qui fait un tracé plus beau visuellement. Cette fonction permet de visualiser le parcours sur une carte.

Au cours de ce projet, j'ai acquis de nouvelles compétences en travaillant avec des données géographiques. En créant les fonctions `graphique_altitude1()`, `graphique_altitude2()` et `graphique_carte()`, j'ai appris à représenter graphiquement ces fonctions à l'aide de la bibliothèque `Matplotlib`. De plus, en travaillant en binôme sur ce projet avec `Léopold`, j'ai su développer ma compétence d'organisation de notre travail. Cette expérience m'a permis d'améliorer ma capacité à travailler en équipe, à échanger des idées et à prendre des décisions à deux.

Compte Rendu de Léopold :

Dans le cadre de ce projet, j'ai travaillé sur le traitement de données géographiques pour un trajet entre Fameck et Florange en France. Plus précisément, j'ai créé une base de données de points de suivi et j'ai écrit des fonctions pour calculer le dénivelé cumulé positif et la distance totale parcourue entre les points.

Pour créer la base de données, j'ai écrit la fonction `creation_base` qui prend en entrée un fichier contenant les données de suivi au format gpx et renvoie une liste contenant les informations de latitude, longitude et altitude pour chaque point de suivi.

La fonction `creation_base(nom_fichier)` prend en entrée le nom d'un fichier contenant des données sur les points de suivi d'un trajet et renvoie une liste contenant les informations de latitude, longitude et altitude pour chaque point de suivi.

Créer une liste vide appelée `base` pour stocker les points de suivi.

Ouvrir le fichier en mode lecture à l'aide de `with open(nom_fichier, 'r')` as `f`:. Cela permet d'ouvrir le fichier spécifié par `nom_fichier` et de le fermer automatiquement une fois le bloc `with` terminé.

Lire le contenu du fichier à l'aide de `contenu = f.read()`.

Diviser le contenu du fichier en une liste de chaînes de caractères, chacune contenant les informations sur un point de suivi, en utilisant `trkpts = contenu.split('<trkpt ')[1:]`. La méthode `split()` divise la chaîne de caractères en utilisant le texte spécifié '`<trkpt '` comme séparateur. La première chaîne de caractères dans la liste résultante ne contiendra pas d'informations sur un point de suivi et sera donc ignorée en utilisant la tranche `[1:]`.

Itérer sur chaque chaîne de caractères `trkpt` de `trkpts` à l'aide de la boucle `for trkpt in trkpts`:.

Extraire la latitude en trouvant l'indice de début de la chaîne '`lat=""`' avec `latitude_start = trkpt.find('lat="') + len('lat="')` et l'indice de fin de la chaîne `""` en utilisant

latitude_end = trkpt.find('"', latitude_start). Ensuite, extraire la sous-chaîne contenant la latitude à l'aide de trkpt[latitude_start:latitude_end] et la convertir en un nombre à virgule flottante avec float(). Stocker le résultat dans la variable latitude.

Extraire la longitude en trouvant l'indice de début de la chaîne 'lon="' avec longitude_start = trkpt.find('lon="') + len('lon="') et l'indice de fin de la chaîne '"' en utilisant longitude_end = trkpt.find('"', longitude_start). Ensuite, extraire la sous-chaîne contenant la longitude à l'aide de trkpt[longitude_start:longitude_end] et la convertir en un nombre à virgule flottante avec float(). Stocker le résultat dans la variable longitude. Extraire l'altitude en trouvant l'indice de début de la chaîne '<ele>' avec altitude_start = trkpt.find('<ele>') + len('<ele>') et l'indice de fin de la chaîne '</ele>' en utilisant altitude_end = trkpt.find('</ele>', altitude_start). Ensuite, extraire la sous-chaîne contenant l'altitude à l'aide de trkpt[altitude_start:altitude_end] et la convertir en un nombre à virgule flottante avec float(). Stocker le résultat dans la variable altitude.

Ajouter les informations de latitude, longitude et altitude à la liste base en créant une nouvelle liste [latitude, longitude, altitude] avec ces informations et en l'ajoutant à la liste base avec base.append([latitude, longitude, altitude]).

Renvoyer la liste base avec return base.

Ensuite, pour calculer le dénivelé cumulé positif, j'ai écrit la fonction denivele_cumule_positif qui prend en entrée la base de données créée précédemment et renvoie le dénivelé cumulé positif.

Enfin, pour calculer la distance totale parcourue entre les points de suivi, j'ai écrit la fonction distance_oiseau qui prend en entrée la base de données et renvoie la distance totale parcourue entre les points.

J'ai rencontré plusieurs difficultés lors de l'analyse de la fonction distance_oiseau. La première difficulté a été de comprendre le calcul de la distance entre deux points en utilisant les coordonnées géographiques de latitude et de longitude.

La deuxième difficulté a été de comprendre comment utiliser les fonctions trigonométriques sin, cos et acos en Python. J'ai dû faire des recherches pour comprendre comment utiliser ces fonctions dans le cadre de la formule de calcul de la distance.

Enfin, la dernière difficulté a été de comprendre comment la boucle for parcourt la base de données. Pour cela j'ai dû faire appel à Romain comme on l'a expliqué dans le compte rendu commun.